

Machine Program: Basics

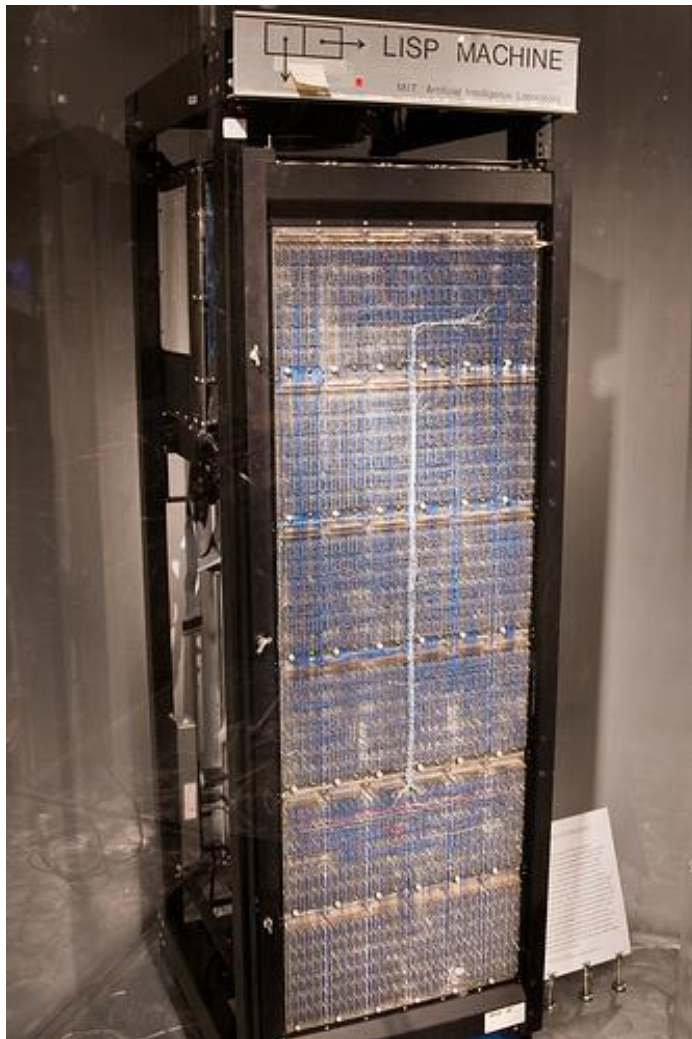
Jinyang Li

Some are based on Tiger Wang's slides

Lesson plan

- What we've learnt so far:
 - How integers/reals/characters are represented by computers
 - C programming
- Today:
 - Basic hardware execution of a program
 - x86 registers
 - x86 move instruction

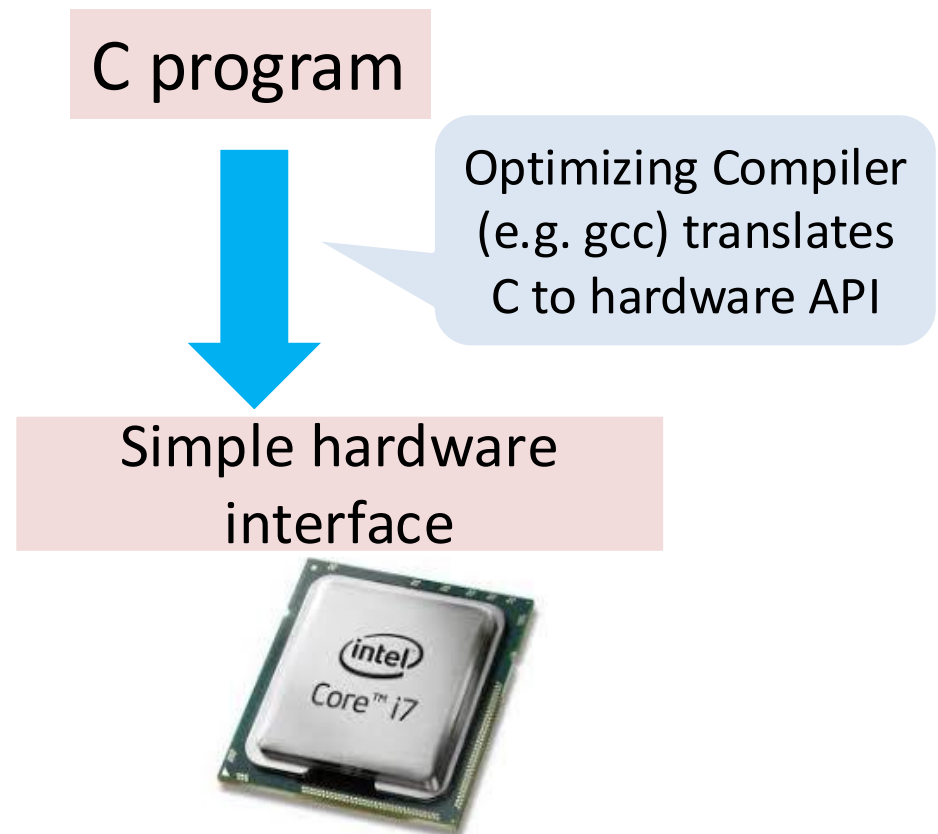
Can we build a machine to execute C directly?



- Historical precedents:
 - LISP machine (80s)
 - Intel iAPX 432 (Ada)

Why not directly execute C?

- Results in very complex hardware design
 - Complex → Hard to implement w/ high performance
- A better approach:



C vs. assembly vs. machine code

C source

x86 assembly

x86 machine code

```
long x;  
long y;  
  
y = x;  
y = 2*y;
```

Compiler

```
movq %rdi, %rax  
addq %rax, %rax
```

assembler

```
01000010000001110  
10001001010100110  
... •
```

gcc -c

does both

gcc -S

compiles to assembly

C vs. machine code

```
long x;  
long y;
```

```
y = x;  
y = 2*y;
```

compile to
x86 machine code



0x00...0058
0x00...0050
0x00...0048
0x00...0040
0x00...0038
y:0x00...0030
0x00...0028
x:0x00...0020
0x00...0018
0x00...0010

...

instruction
instruction
instruction
data
data
data
data
.....

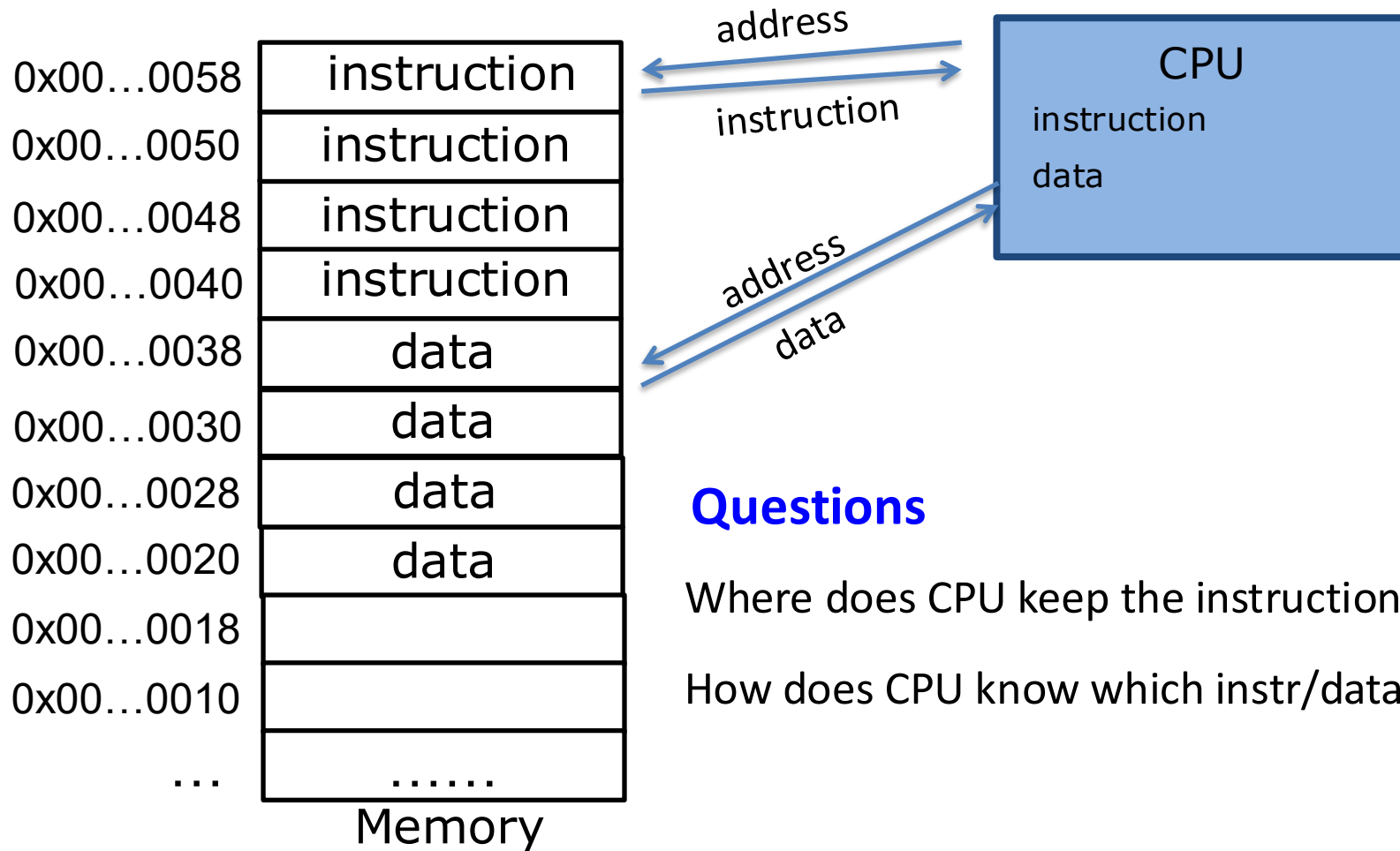
Memory

E.g. move data
from one
memory location
to another

E.g. multiply the
number at some
memory location
by a constant

No concept of
variables,
scopes, types

How CPU executes a program

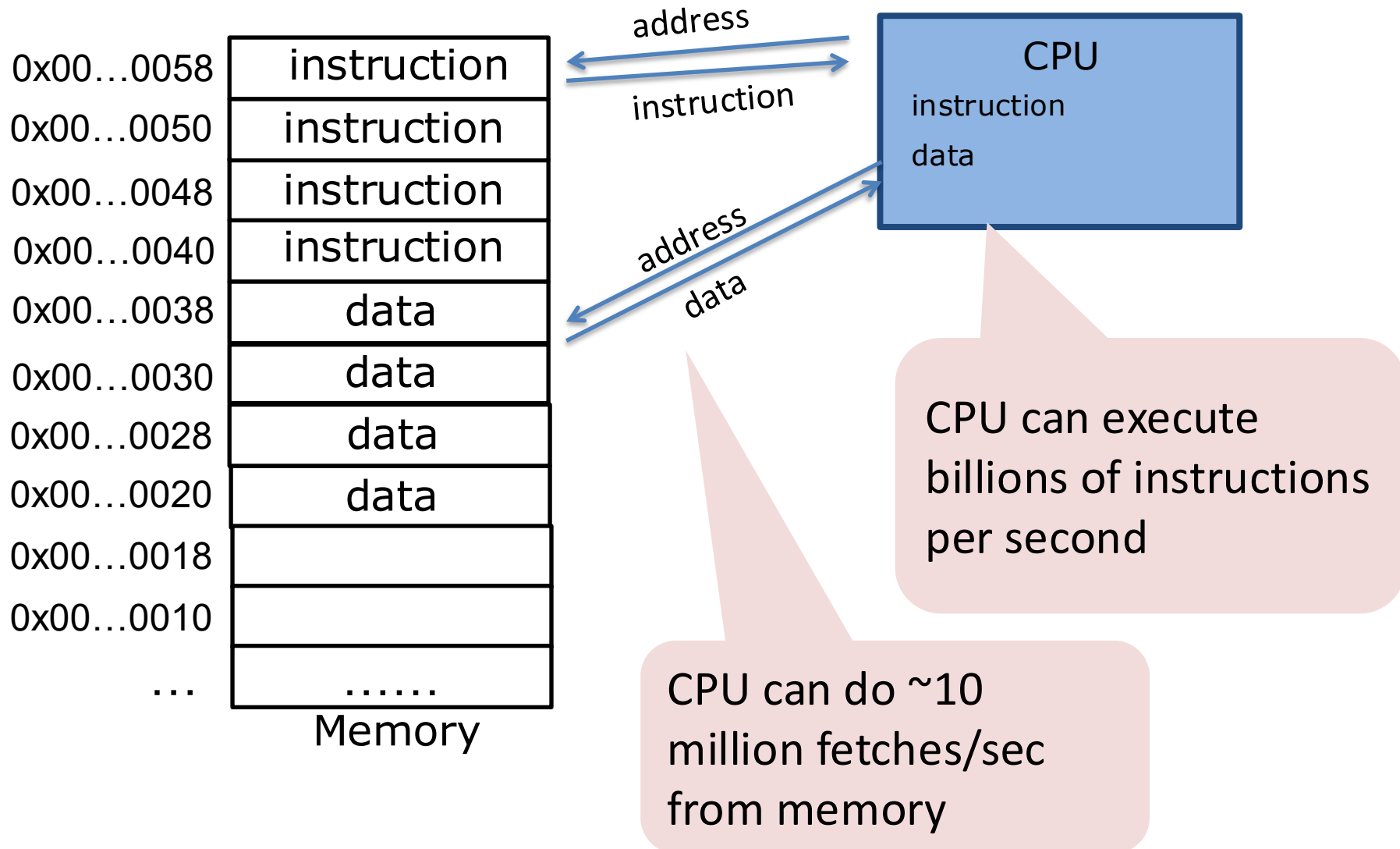


Questions

Where does CPU keep the instruction and data?

How does CPU know which instr/data to fetch?

How CPU executes a program



Register – temporary storage area built into a CPU

PC: Program counter, also called instruction pointer (IP).

- Store memory address of next instruction

IR: CPU's internal buffer storing the fetched instruction

General purpose registers:

- Store data and address used by programs

Program status and control register:

- Status of the instruction executed

Steps of execution in CPU

1. PC contains the instruction's address
2. Fetch the instruction to internal buffer
3. Execute the instruction which does one of following:
 - Memory operations: move data from memory to register (or opposite)
 - Arithmetic operations: add, shift etc.
 - Control flow operations.
4. PC is updated to contain the next instruction's address.

Instruction Set Architecture (ISA)

- ISA: interface exposed by hardware to software writers

- **X86_64** is the ISA implemented by Intel/AMD CPUs
 - 64-bit version of x86

Lectures on assembly

- ARM is another common ISA
 - Phones, tablets, Raspberry Pi, Apple's new M1 laptop

- **RISC-V** is yet another ISA
 - P&H textbook's ISA.
 - Open-sourced, royalty-free

Lectures on hardware

Apple laptops use ARM chips. Hence we need a virtual machine to run C programs compiled to x86

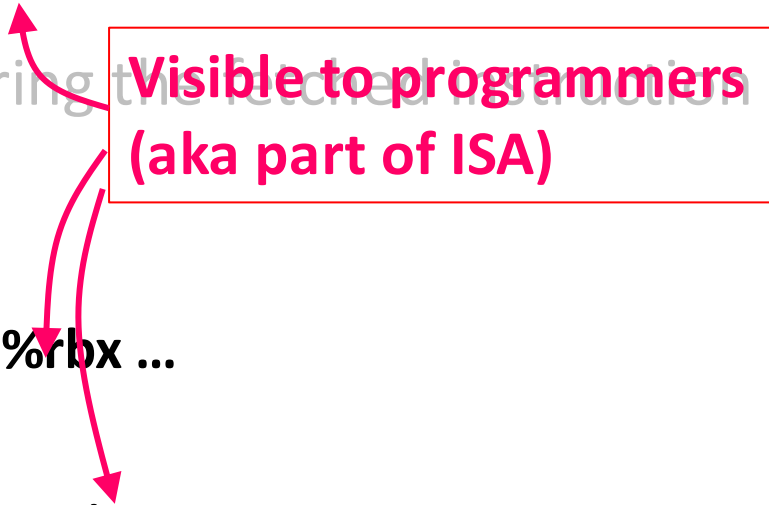
X86-64 ISA: registers

Program counter:

- called `%rip` in `x86_64`

IR: CPU's internal buffer storing the fetched instruction

Visible to programmers
(aka part of ISA)



General purpose registers:

- 16 8-byte registers: `%rax`, `%rbx` ...

Program status and control register:

- Called “RFLAGS” in `x86_64`

X86-64 general purpose registers: 8-byte

%rax	%r8
%rbx	%r9
%rcx	%r10
%rdx	%r11
%rsi	%r12
%rdi	%r13
%rsp	%r14
%rbp	%r15



8 bytes

X86-64 general purpose registers: 4-byte

4-byte registers refer to the lower-order 4-bytes of original registers.

%eax refers to the lower-order
4-byte of %rax

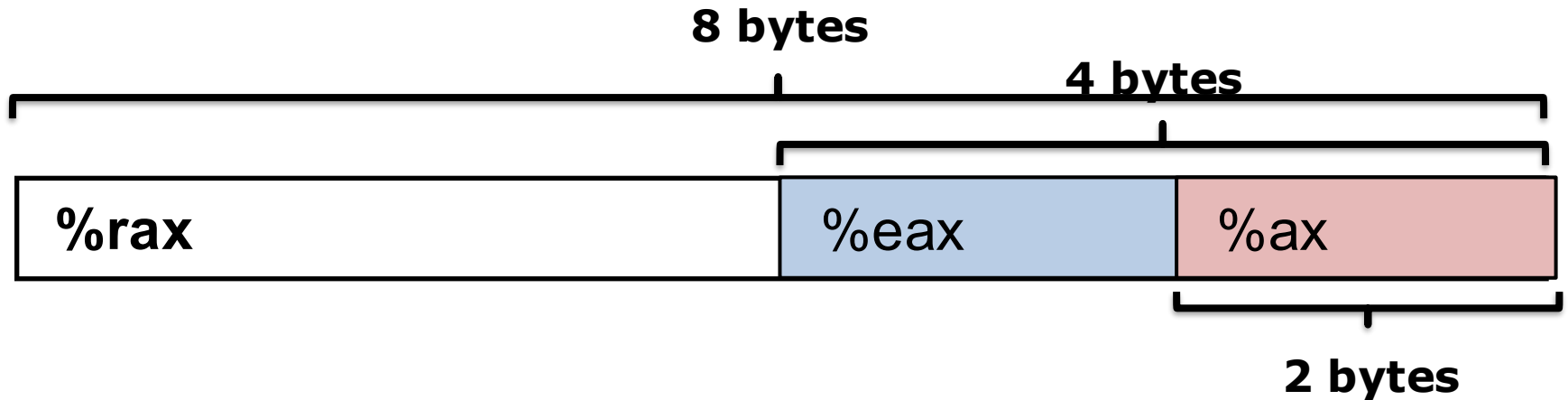
%rax	%eax	%r8	%r8d
%rbx	%ebx	%r9	%r9d
%rcx	%ecx	%r10	%r10d
%rdx	%edx	%r11	%r11d
%rsi	%esi	%r12	%r12d
%rdi	%edi	%r13	%r13d
%rsp	%esp	%r14	%r14d
%rbp	%ebp	%r15	%r15d

8 bytes

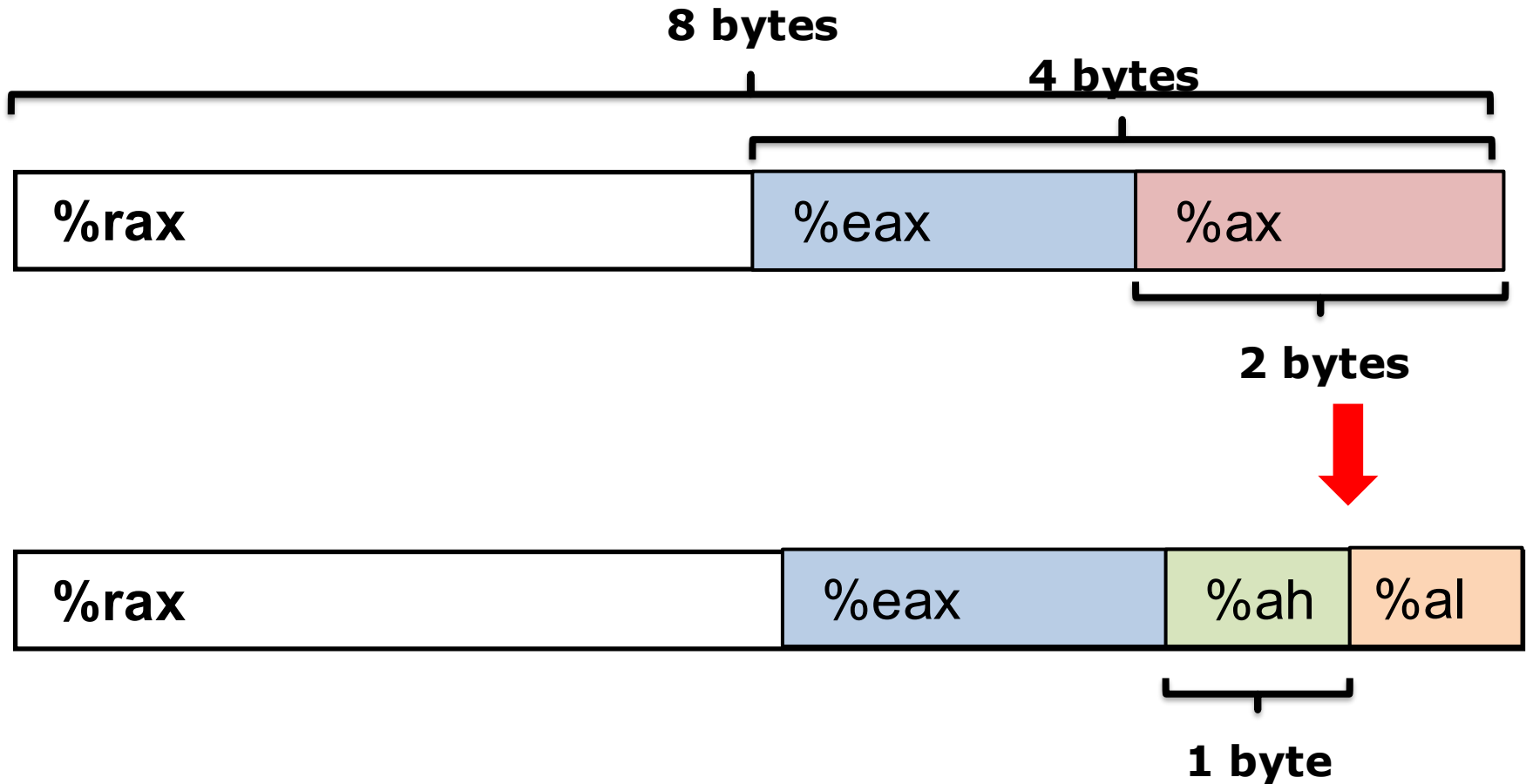
4 bytes

X86-64 general purpose registers: 2-byte

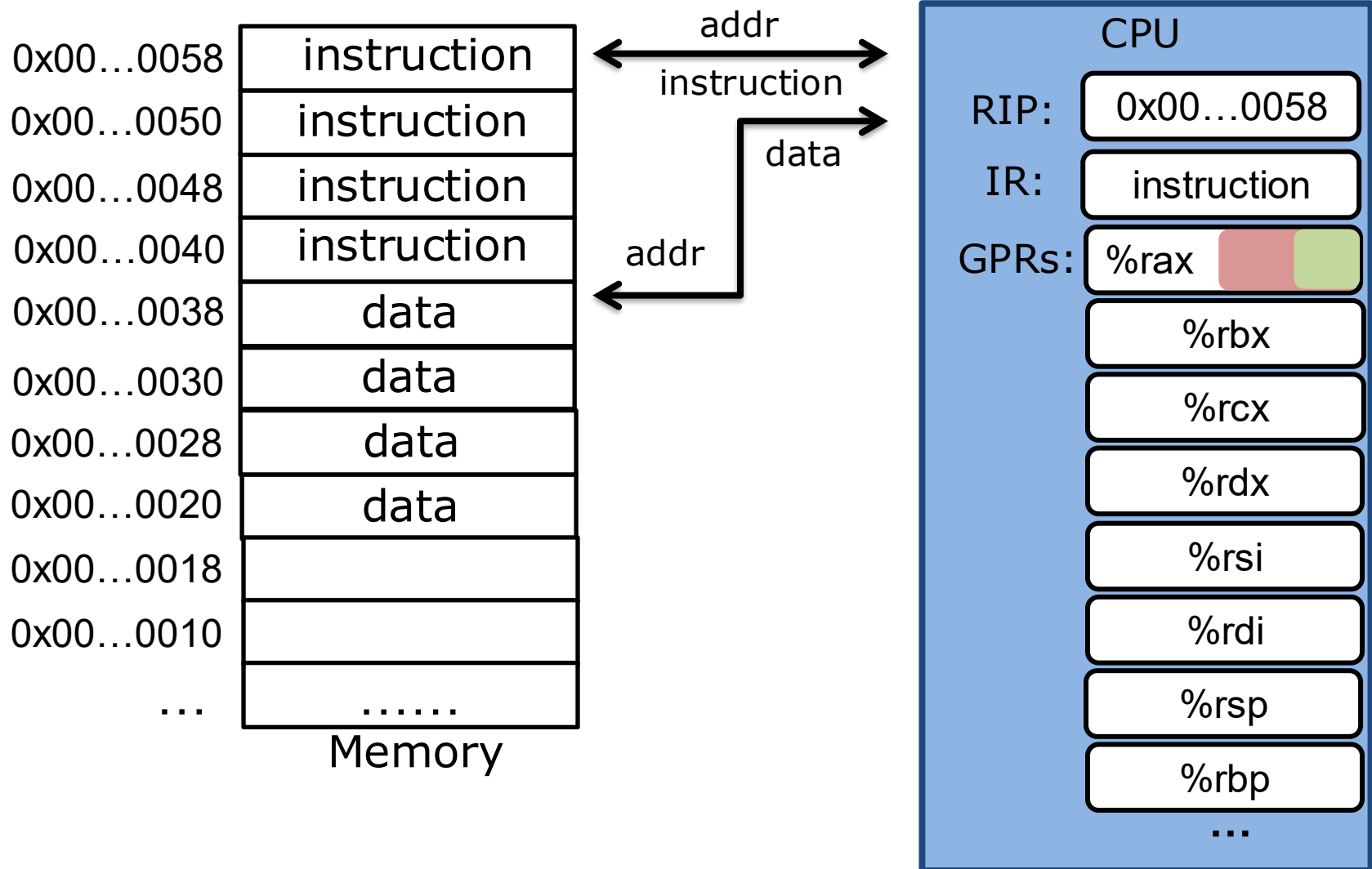
2-byte registers refer to the lower-order 2-bytes of original registers.



X86-64 general purpose registers: 1-byte



x86-64 execution



X86 ISA



Intel® 64 and IA-32 Architectures Software Developer's Manual

**Combined Volumes:
1, 2A, 2B, 2C, 2D, 3A, 3B, 3C, 3D and 4**

NOTE: This document contains all four volumes of the Intel 64 and IA-32 Architectures Software Developer's Manual: Basic Architecture, Order Number 253665; Instruction Set Reference A-Z, Order Number 325383; System Programming Guide, Order Number 325384; Model-Specific Registers, Order Number 335592. Refer to all four volumes when evaluating your design needs.

Order Number: 325462-065US
December 2017

A must-read for
compiler and OS writers

<https://software.intel.com/en-us/articles/intel-sdm#combined>